



AUDIT REPORT

May 2026

For



Table of Content

Executive Summary	03
Number of Security Issues per Severity	05
Summary of Issues	06
Checked Vulnerabilities	07
Techniques and Methods	09
Types of Severity	11
Types of Issues	12
Severity Matrix	13
 Medium Severity Issues	14
1. Signature Missing Replay Domain Parameters	14
 Low Severity Issues	15
2. Signatures Do Not Use EIP-712 Typed Structured Data	15
 Informational Issues	16
3. Missing Events on Critical Admin Updates	16
4. Native Token Withdrawal Can Fail if Receiver Rejects Native Token	17
5. Missing Explicit Zero Address Check for Owner	18
6. Use of `OwnableUpgradeable` Instead of `Ownable2StepUpgradeable`	19
Functional Tests	20
Threat Model	21
Automated Tests	30
Closing Summary & Disclaimer	31



Executive Summary

Project Name	Watt2Trade
Protocol Type	Presale
Project URL	https://www.watt2trade.com/
Overview	An upgradeable presale contract designed to collect payments from buyers in POL, USDC, and USDT. Each purchase must be authorized through a backend-issued ECDSA signature, allowing the project to control eligible buyers, approved purchase amounts, and replay protection through user-specific nonces. The contract includes administrative controls for pausing purchases, updating supported token addresses, managing the authorized signer and receiver address, and withdrawing collected funds.
Audit Scope	The scope of this Audit was to analyze the The watt2trade Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/Watt-2-Trade/Presale-/blob/main/PresaleV1.sol
Branch	main
Contracts in Scope	PresaleV1.sol
Commit Hash	10c59cbbce7f6a82a64543dd23f89cab99f8d6d6
Language	Solidity
Blockchain	Polygon
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	18th May 2026 - 19th May 2026
Updated Code Received	29th May
Review 2	29th May
Fixed In	1bc197a84dd74e66303f000fe099211ff76d2c7



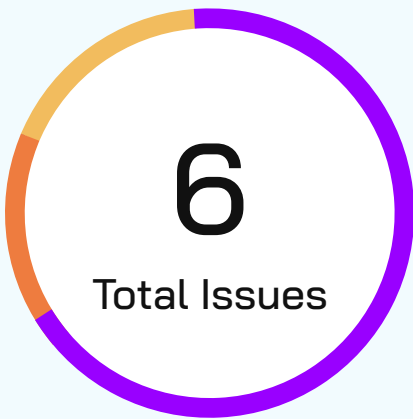
Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>

.



Number of Issues per Severity



Critical	0 (0%)
High	0 (0%)
Medium	1 (16.66%)
Low	1 (16.66%)
Informational	4 (66.66%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	1	2
	Partially Resolved	0	0	0	0	0
	Resolved	0	0	1	0	2



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Signature Missing Replay Domain Parameters	Medium	Resolved
2	Signatures Do Not Use EIP-712 Typed Structured Data	Low	Acknowledged
3	Missing Events on Critical Admin Updates	Informational	Resolved
4	Native Token Withdrawal Can Fail if Receiver Rejects Native Token	Informational	Acknowledged
5	Missing Explicit Zero Address Check for Owner	Informational	Acknowledged
6	Use of OwnableUpgradeable Instead of Ownable2StepUpgradeable	Informational	Resolved



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ **Missing Zero Address Validation**

✓ **Private modifier**

✓ **Revert/require functions**

✓ **Multiple Sends**

✓ **Using suicide**

✓ **Using delegatecall**

✓ **Upgradeable safety**

✓ **Using throw**

✓ **Using inline assembly**

✓ **Style guide violation**

✓ **Unsafe type inference**

✓ **Implicit visibility level**

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



Medium Severity Issues

Signature Missing Replay Domain Parameters

Resolved**Path**

PresaleV1.sol

Function Name`recoverSigner`**Description**

The signed payload only includes the amount, user address, and nonce. It does not include important domain and context fields such as chain ID, verifying contract address, payment token, or deadline. Because of this, a signature may be valid in more than one context if another contract uses the same signer and nonce structure. The same signature could potentially be replayed across deployments, chains, or token purchase contexts depending on how the backend manages signatures.

Impact

Medium

Likelihood

Medium

Recommendation

Bind signatures to a specific domain and purchase context. The signed data should include the chain ID, the contract address, the selected payment token, the buyer address, the amount, the nonce, and an expiry deadline.



Low Severity Issues

Signatures Do Not Use EIP-712 Typed Structured Data

Acknowledged**Path**

PresaleV1.sol

Function Name`recoverSigner`**Description**

The contract verifies signatures using an Ethereum signed message hash built from packed values. This approach produces an opaque message for signers and does not provide the same readability and structure as EIP-712 typed data. With the current design, the signer may not clearly see the exact purchase fields being approved in a wallet or signing interface. EIP-712 is the standard approach for structured signatures because it allows clear typed fields and a domain separator.

Impact

Low

Likelihood

Medium

Recommendation

Use EIP-712 typed structured data for purchase approvals. The signed message should clearly define fields such as buyer address, payment token, amount, nonce, deadline, chain ID, and verifying contract. This improves readability for signers and makes the authorization format easier to validate and maintain.

Client's Take

Signatures are generated server-side by the backend, not by user wallets. EIP-712 flow is not required as there is no direct user signing interaction.

Status: Not applicable to current design



Informational Severity Issues

Missing Events on Critical Admin Updates

Resolved**Path**

PresaleV1.sol

Function Name`updateApprover`, `updateReceiver`**Description**

The `updateApprover` and `updateReceiver` functions allow the owner to update critical protocol configuration. The `updateApprover` function changes the trusted signer used to authorize user purchases, while the `updateReceiver` function changes the address that receives withdrawn POL and ERC20 tokens. Both of these are sensitive administrative actions because they directly affect purchase authorization and fund flow. However, neither function emits an event when the corresponding address is updated. Without events, off-chain monitoring systems, indexers, users, and auditors have reduced visibility into these important changes

Impact

Low

Likelihood

Low

Recommendation

Add dedicated events for both approver and receiver updates, and emit them whenever these values are changed. Each event should include the previous address and the new address so off-chain systems can maintain a clear history of administrative changes.



Native Token Withdrawal Can Fail if Receiver Rejects Native Token

Acknowledged

Path

PresaleV1.sol

Function Name

`withdrawEth()`

Description

`withdrawEth()` sends POL to the configured receiver via a low-level call. If receiver is ever updated to a smart contract lacking a payable receive/fallback, or one whose logic reverts, the withdrawal will fail with `TransferFailed`. Funds are not permanently locked since the owner can call `updateReceiver()`, but withdrawals remain blocked until that action is taken.

Impact

Low

Likelihood

Low

Recommendation

Ensure that the configured receiver address can successfully receive POL before using it in production.



Missing Explicit Zero Address Check for Owner

Acknowledged

Path

PresaleV1.sol

Function Name

`initialize`

Description

The initializer validates `_usdc`, `_usdt`, `_approver`, and `_receiver` against the zero address, but it does not explicitly validate `_owner`. The owner is a critical role because it controls pausing, token updates, signer updates, receiver updates, and withdrawals. Even if the inherited OpenZeppelin initializer may reject a zero owner depending on the library version, relying only on inherited behavior makes the validation less clear in this contract.

Impact

Low

Likelihood

Low

Recommendation

Add an explicit zero-address check for `_owner` inside the initializer together with the other initialization checks. This makes the contract's assumptions clear and avoids depending on external library behavior for such a critical role.

Client's Take

Deployment is script-driven with a manual checklist review. Passing `address(0)` as owner is managed in the deployment script.



Use of `OwnableUpgradeable` Instead of `Ownable2StepUpgradeable`

Resolved

Path

PresaleV1.sol

Description

The contract uses OpenZeppelin's OwnableUpgradeable module for access control. With this module, ownership is transferred in a single transaction by the current owner. This means the new owner address becomes active immediately after the transfer call succeeds. If the current owner accidentally transfers ownership to an incorrect address, an address with no private key access, or a contract that does not support ownership management, the contract may permanently lose administrative control.

Since this contract depends heavily on owner-only functions for pausing, updating token addresses, updating the signer, updating the receiver, and withdrawing funds, ownership transfer should be handled with extra safety.

Impact

Low

Likelihood

Low

Recommendation

Use Ownable2StepUpgradeable so the new owner must explicitly accept ownership.



Functional Tests

Some of the tests performed are mentioned below:

- ✓ Verified that users can purchase using POL when the sent value matches the approved deposit amount.
- ✓ Verified that users can purchase using supported ERC20 tokens, including USDC and USDT.
- ✓ Verified that token purchases require a valid backend-issued ECDSA signature.
- ✓ Verified that reused signatures are prevented through user-specific nonce incrementation.
- ✓ Verified that purchases cannot be executed while the contract is paused.
- ✓ Verified that only the owner can pause and unpaue the contract.
- ✓ Verified that only the owner can withdraw collected POL.
- ✓ Verified that only the owner can withdraw collected USDC/USDT tokens.
- ✓ Verified that withdrawals revert when the requested amount exceeds the contract balance.
- ✓ Verified that the owner can update supported token addresses.
- ✓ Verified that the owner can update the approver address.
- ✓ Verified that the owner can update the receiver address.



Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

VotingEscrow

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IERC20 OpenZeppelin	Interface	Defines the ERC20 interface used for USDC and USDT interactions	Token contracts follow the ERC20 standard	Non-standard token behavior may break payment accounting
SafeERC20 OpenZeppelin	Library	Wraps ERC20 transfers with safer handling through safeTransfer and safeTransferFrom	Library correctly handles missing/false return values and reverts on failure	If token behavior is non-standard beyond return values, transfers may still create accounting mismatch
ECDSA OpenZeppelin	Library	Recovers the signer from the purchase authorization signature	Signature recovery is correctly implemented	Incorrect signing scheme or missing domain fields can allow replay or signer confusion

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
OwnableUpgradeable OpenZeppelin	Base Contract	Provides owner-only access control for admin functions	Owner is trusted and operationally secure	Single-step ownership transfer can lead to accidental or permanent ownership loss
PausableUpgradeable OpenZeppelin	Base Contract	Allows owner to pause and unpause purchase functions	Pause logic works correctly and owner responds when needed	Centralized owner can stop user purchases
ReentrancyGuardUpgradeable OpenZeppelin	Base Contract	Protects purchase and withdrawal functions using nonReentrant	Reentrancy mutex works correctly	N.A
Initializable OpenZeppelin	Base Contract	Replaces constructor logic for proxy deployments	Initializer is called correctly and only once	Incorrect initialization may leave contract misconfigured
USDC	External ERC20 Contract	Accepted as a payment token in buyWithToken and withdrawable through withdrawToken	Token behaves like standard USDC with no fee-on-transfer behavior	Blacklisting, pausing, or non-standard behavior can cause transfer failures
USDT	External ERC20 Contract	Accepted as a payment token in buyWithToken and withdrawable through withdrawToken	Token behaves like standard USDT with no fee-on-transfer behavior	Blacklisting, pausing, or non-standard behavior can cause transfer failures



Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Backend Approver	Off-chain signer	Signs purchase authorizations used by buyWithEth and buyWithToken	Approver key is secure and signs only valid purchases	Compromised signer can authorize unwanted purchases
Receiver Address	External address	Receives withdrawn POL, USDC, and USDT	Receiver is controlled by the project and can receive POL/tokens	If receiver rejects POL, POL withdrawal can fail

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
PresaleV1.sol	initialize()	What this function can do Set USDC, USDT, approver, receiver and owner once What this function cannot/should not do Be called again after deployment. Accept zero addresses Main invariant(s) All addresses non-zero after init. Runs only once Access level Public but initializer – once only



Contract Name	Function	Category
PresaleV1.sol	buyWithEth()	What this function can do Accept POL, verify approver signature, increment nonce, emit event What this function cannot/should not do Accept POL without valid signature. Allow zero value. Replay same signature only checks whether msg.value equals _depositAmount. It does not explicitly reject _depositAmount == 0. Main invariant(s) msg.value == _depositAmount. Nonce increments exactly once per valid sig Access level Public, whenNotPaused, nonReentrant
	buyWithToken()	What this function can do Pull USDC or USDT from user, verify signature, increment nonce What this function cannot/should not do Accept zero amount. Use POL signature for token purchase. Allow replay Main invariant(s) Transfer only succeeds with valid sig. Nonce increments exactly once Access level Public, whenNotPaused, nonReentrant



Contract Name	Function	Category
PresaleV1.sol	withdrawToken()	What this function can do Transfer USDC or USDT contract balance to receiver What this function cannot/should not do Withdraw more than balance. Withdraw stranded tokens after updateToken Main invariant(s) amount <= contract token balance Access level onlyOwner, nonReentrant
	withdrawEth()	What this function can do Send POL from contract to receiver What this function cannot/should not do Withdraw more than balance. Fail if receiver rejects POL Main invariant(s) amount <= address(this).balance Access level onlyOwner, nonReentrant

Contract Name	Function	Category
	updateToken()	What this function can do Replace USDC and USDT addresses with new ones What this function cannot/should not do Set zero addresses. Silently strand deposits at old token addresses The initialize function accepts token addresses in the order _usdc, _usdt, but updateToken accepts them in the order _usdt, _usdc. This inconsistency can confuse operators or deployment scripts. Main invariant(s) USDC and USDT must always be valid non-zero addresses Access level onlyOwner
	updateApprover()	What this function can do Replace the signature approver address What this function cannot/should not do Set zero address. Change silently without emitting event Main invariant(s) Approver must always be a valid non-zero address Access level onlyOwner

Contract Name	Function	Category
	updateReceiver()	What this function can do Replace the fund receiver address What this function cannot/should not do Set zero address. Set non-payable contract. Change without emitting event Main invariant(s) Receiver must always be a valid payable address Access level onlyOwner
	pause() / unpause()	What this function can do Halt or resume buyWithEth and buyWithToken What this function cannot/should not do Pause admin functions like withdraw. Be called by non-owner Main invariant(s) Only owner can toggle. Withdrawals unaffected by pause Access level onlyOwner

Contract Name	Function	Category
	getNonce()	What this function can do Return current nonce for any user address What this function cannot/should not do Return stale value Main invariant(s) Must match nonces mapping exactly Access level Public view — redundant, nonces is already public

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
PresaleV1.sol	Native Token	Native Token paid by users via buyWithEth()
PresaleV1.sol	ERC20 (USDC)	USDC paid by users via buyWithToken()
PresaleV1.sol	ERC20 (USDT)	USDT paid by users via buyWithToken()



3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
PresaleV1.sol	buyWithEth()	POL (native)	–	Public	Zero amount not blocked. Signature missing chainId, contract address, and token type cross-deployment & cross-function replay possible
PresaleV1.sol	buyWithToken()	USDC / USDT	–	Public	Token type excluded from signature – POL signature reusable here. CEI violation (nonce incremented after transfer)
PresaleV1.sol	withdrawToken()	–	USDC / USDT	onlyOwner	Tokens stranded if updateToken() changes address before withdrawal. Receiver is fixed no destination override
PresaleV1.sol	withdrawEth()	–	POL (native)	onlyOwner	Receiver may reject POL if it is a contract (e.g. multisig without fallback). Receiver is fixed no destination override



4. Design Scope & Assumptions

Item	Detail
Contract Purpose	Payment collection only. Funds held in contract until owner withdraws after presale phases conclude
Contract Owner	Set to Watt2Trade's wallet
Token Distribution	Never mints or transfers sale tokens. Airdrop executed off-chain after TGE based on on-chain + off-chain transaction records
Transaction Recording	Backend RPC listener records all transactions live
Token Price Logic	No on-chain pricing. Price determined off-chain by the approver before signing
Raise Cap	No on-chain hardcap/softcap. Controlled off-chain by stopping signature issuance
Vesting / Cliff	Out of scope
Refund Mechanism	No refunds. Purchases are final once accepted
Presale Approach	Intentionally "light", no vesting mechanics or full presale platform features

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Closing Summary

In this report, we have considered the security of The Watt2Trade (Presale). We performed our audit according to the procedure described above.

No Critical or High severity issues were identified during the audit. The review found 6 issues in total, consisting of 1 Medium severity issue, 1 Low severity issue, and 4 Informational severity issues.

Few of the issues were resolved and others were acknowledged.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1500+

Projects Secured

Follow Our Journey



AUDIT REPORT

May 2026

For



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com